

Real-Data PID Pipeline Plan

Living design document for comparing DNN model contributions to neural responses

Version 0.1 - working draft

1. Purpose and scope

This document defines the first working design for a real-data analysis pipeline using Partial Information Decomposition (PID) to compare the contribution of different deep learning models to explaining neural responses. The document is intended to be used as a shared source for future planning and coding conversations.

The central software principle is that the pipeline should be PID-agnostic. The pipeline should not implement one specific PID definition internally. Instead, it should receive a PID calculation function from the outside, call it using a standardized interface, and store its returned components in a common result schema.

The scientific goal is to ask whether two model-derived representations or predictions contain redundant, unique, or synergistic information about neural responses. The exact PID definition, dimensionality reduction method, covariance estimator, and uncertainty method should remain replaceable.

2. Core design principles

- Separate the scientific question from the software implementation.
- Keep the PID definition external and replaceable.
- Make feature manipulation modular, so PCA or future methods can be inserted without changing the rest of the code.
- Always compute mutual information and diagnostic quantities separately from the PID function.
- Save enough metadata to reconstruct every analysis decision.
- Treat real-data analysis as an estimation problem, not only a computation problem.
- Make every output a long-format row that can later be grouped, filtered, plotted, and compared.

3. Main random variables

The intended bivariate PID structure is:

```
Target:   T = neural responses
Source 1: X1 = representation or prediction from model 1
Source 2: X2 = representation or prediction from model 2
```

Two possible interpretations must be kept separate:

- Representation-level PID: X1 and X2 are raw or transformed DNN features, while T is neural activity.
- Prediction-level PID: X1 and X2 are cross-validated predictions of the neural responses from two encoding models, while T is the measured neural activity.

The current preference is prediction-level PID when feasible, because X1, X2, and T then live in a comparable neural-response space. However, the pipeline should support both modes.

4. Pipeline layers

Layer 1: Data loading

Load and organize raw neural data, DNN model features, prediction files, and metadata. This layer should not perform PCA, regression, MI, or PID.

Recommended outputs:

- neural responses
- model features or model predictions
- image or trial metadata
- subject, area, session, layer, and model identifiers

Layer 2: Preprocessing

Perform operations that clean and align the data before analysis. This includes sample alignment, z-scoring, bad-unit removal, repeat averaging, and missing-value handling.

Recommended outputs:

- number of samples before and after cleaning
- number of features before and after cleaning
- excluded units or images
- whether z-scoring and repeat averaging were applied

Layer 3: Feature manipulation

Apply optional transformations to sources and target. This layer should support identity transforms, PCA, random projections, CCA-like methods, or future transformations.

Recommended outputs:

- transform name
- number of components
- explained variance
- effective dimension before and after transformation
- whether the transform was fitted inside cross-validation

Layer 4: Estimation and MI

Estimate covariance and compute classical information-theoretic quantities that are independent of the chosen PID definition.

Recommended outputs:

- $I(T;X1)$
- $I(T;X2)$
- $I(T;X1,X2)$
- $I(X1;X2)$
- conditional MI values
- co-information
- covariance condition number

Layer 5: PID callable

Call an external PID function using a standardized interface. The pipeline should not know whether the function implements Gaussian BROJA, flow-based BROJA, delta-PID, Idep, or another future definition.

Recommended outputs:

- redundancy
- unique information of X1
- unique information of X2
- synergy
- method name
- method-specific diagnostics

Layer 6: Results and diagnostics

Store results, metadata, sanity checks, uncertainty estimates, and diagnostic flags in a structured format suitable for plotting and later analysis.

Recommended outputs:

- PID identity errors
- negative-component flags
- singular-covariance flags
- bootstrap intervals
- permutation baselines
- predictive performance metrics

5. Standard PID function interface

Each PID implementation should be wrapped to match the same interface. A suggested interface is:

```
def pid_func(target, source_1, source_2, *, estimator_context):
    return {
        'red': float,
        'unq1': float,
        'unq2': float,
        'syn': float,
        'method': str,
        'extra': dict,
    }
```

The pipeline should only require the four standard PID components. All method-specific quantities should be stored under the extra field, without being required by the rest of the pipeline.

6. Minimal result schema

Each analysis should produce one row or one structured dictionary with the following groups:

```
result = {
  'metadata': {...},
  'pid': {
    'red': ...,
    'unq1': ...,
    'unq2': ...,
    'syn': ...,
  },
  'mi': {
    'I_T_X1': ...,
    'I_T_X2': ...,
    'I_T_X1X2': ...,
    'I_X1_X2': ...,
    'co_info': ...,
  },
  'normalized': {
    'red_frac': ...,
    'unq1_frac': ...,
    'unq2_frac': ...,
    'syn_frac': ...,
  },
  'diagnostics': {
    'pid_identity_error_joint': ...,
    'pid_identity_error_x1': ...,
    'pid_identity_error_x2': ...,
    'min_eigenvalue_cov': ...,
    'condition_number_cov': ...,
    'n_samples': ...,
    'dim_T': ...,
    'dim_X1': ...,
    'dim_X2': ...,
  }
}
```

7. Quantities beyond PID and MI

The pipeline should eventually output additional quantities that help interpret and validate PID results.

Quantity	Purpose	Examples
Predictive performance	Check whether information-theoretic results agree with encoding-model behavior.	$R2(T \sim X1)$, $R2(T \sim X2)$, $R2(T \sim X1 + X2)$
Reliability / noise ceiling	Interpret PID values relative to the reliability of the neural target.	split-half reliability, trial variance, repeat consistency
Uncertainty	Assess whether components are stable across samples.	bootstrap mean, standard error, 95 percent CI
Permutation baselines	Test whether apparent information survives null controls.	shuffle target, shuffle source, image-label permutation
Dimensionality diagnostics	Track whether high-dimensional estimation is reliable.	effective dimension, PCA variance, $n_samples / dimension\ ratio$
Numerical stability	Detect covariance and optimization failures.	condition number, minimum eigenvalue, convergence status

8. Sanity checks

For every run, the pipeline should automatically check the bivariate PID identities:

$$\begin{aligned} I(T; X1, X2) &= Red + Unq1 + Unq2 + Syn \\ I(T; X1) &= Red + Unq1 \\ I(T; X2) &= Red + Unq2 \end{aligned}$$

The pipeline should flag the following cases:

- any PID component is negative beyond a numerical tolerance
- the PID identities do not close within tolerance
- the covariance matrix is singular or ill-conditioned
- the number of samples is too small relative to the total dimension
- the chosen PCA dimension explains too little variance
- bootstrap confidence intervals are very wide
- unique information changes qualitatively under small changes in preprocessing or dimension

9. Minimal first implementation

The first version should be intentionally small:

- Input T, X1, and X2 for one subject, one brain area, one pair of models, and one layer pair.
- Align samples and remove invalid rows.
- Z-score features.
- Apply optional PCA or identity transform.
- Estimate covariance.
- Compute MI values.
- Call the external PID function.
- Check PID identities and numerical diagnostics.
- Save one long-format result row.

Only after this works should the pipeline expand to loops over subjects, brain areas, time windows, models, layers, PCA dimensions, and PID definitions.

10. Suggested high-level API

```
result = run_pid_comparison(
    target=T,
    source_1=X1,
    source_2=X2,
    pid_func=broja_gaussian_pid,
    source_transform=PCA(n_components=20),
    target_transform=PCA(n_components=20),
    mi_estimator=gaussian_mi,
    covariance_estimator=LedoitWolf(),
    bootstrap=None,
    metadata={
        'subject': 'S1',
        'area': 'IT',
        'model_1': 'CLIP',
        'layer_1': 'visual.layer_10',
        'model_2': 'DINOv2',
        'layer_2': 'blocks.11',
    },
)
```

11. Open decisions for the next planning conversation

- Is the target variable T trial-level neural responses, image-averaged neural responses, or responses after an encoding model?
- Should X1 and X2 be raw DNN features or cross-validated neural predictions from each model?
- Should the first real-data version use PCA on sources only, target only, or both?
- Which covariance estimator should be the first default: sample covariance, Ledoit-Wolf, OAS, or another estimator?
- Should uncertainty begin with bootstrap, permutation, or both?

- Should results be stored as CSV, Parquet, JSON, or all three?
- What is the first smallest scientifically meaningful comparison: two models, two layers, one subject, one area?

12. Future extensions

- Cross-validated prediction-level PID, where X_1 and X_2 are held-out predictions of T .
- Layer-by-layer model comparison heatmaps.
- Model-family summaries: architecture, training objective, training data, and scale.
- Stability analysis across PCA dimension.
- Permutation-based lower bounds for unique information.
- Comparison of multiple PID definitions under the same preprocessing and MI estimation pipeline.
- Optional storage of transformed features and covariance matrices for reproducibility.

13. Working convention for future conversations

When referring to this document in future conversations, “the real-data PID pipeline” means the modular pipeline described here. The default assumption is that the pipeline is PID-agnostic, transform-agnostic, and estimator-aware. Any new code should preserve these separations unless there is an explicit reason to simplify them.

The next development step is not to implement every feature. The next step is to choose the first minimal data format and decide whether the first analysis should operate on DNN features or on cross-validated neural predictions.